Industrial Internship Report on

”File Organizer in Python”

Prepared by

[Koyana rathore]

Executive Summary
This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks’ time. My project was (Tell about ur Project) This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1 Preface 3

2 Introduction 4

2.1 About UniConverge Technologies Pvt Ltd 4

2.2 About upskill Campus 8

2.3 Objective 10

2.4 Reference 10

2.5 Glossary 10

3 Problem Statement 11

4 Existing and Proposed solution 12

5 Proposed Design/ Model 13

5.1 High Level Diagram (if applicable) .... **Error! Bookmark not defined.**

5.2 Low Level Diagram (if applicable) .... **Error! Bookmark not defined.**

5.3 Interfaces (if applicable) .... **Error! Bookmark not defined.**

6 Performance Test 15

6.1 Test Plan/ Test Cases .... **Error! Bookmark not defined.**

6.2 Test Procedure .... **Error! Bookmark not defined.**

6.3 Performance Outcome .... **Error! Bookmark not defined.**

7 My learnings 15

8 Future work scope .... **Error! Bookmark not defined.**

1 Preface

Executive Summary

This report documents the development of File Organizer, a desktop application implemented in Python and presented in the code as SmartSort. The project addresses the common problem of cluttered folders, where documents, images, audio, videos, archives, installers, and source-code files are mixed together. Finding a particular file in such folders can take time and makes routine file management less convenient.

The application provides a graphical interface that allows a user to select a folder, preview how files will be classified, and then confirm the organization process. Files are grouped into category folders according to their extensions. The implementation also includes safeguards against overwriting files with the same name, a record of file movements for undoing the latest batch, and a summary of the operation.

The project uses Python's standard-library modules, including Tkinter for the interface, pathlib for path handling, shutil for moving files, json for saving operation history, and datetime for recording timestamps. The report explains the problem, proposed design, implementation approach, testing plan, learning outcomes, and possible future improvements. Test outcomes should be updated with actual results from running the application on the user's system.

Prepared by: [KOYANA RATHORE]

Internship provider: upskill Campus, with The IoT Academy and UniConverge Technologies Pvt. Ltd.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.



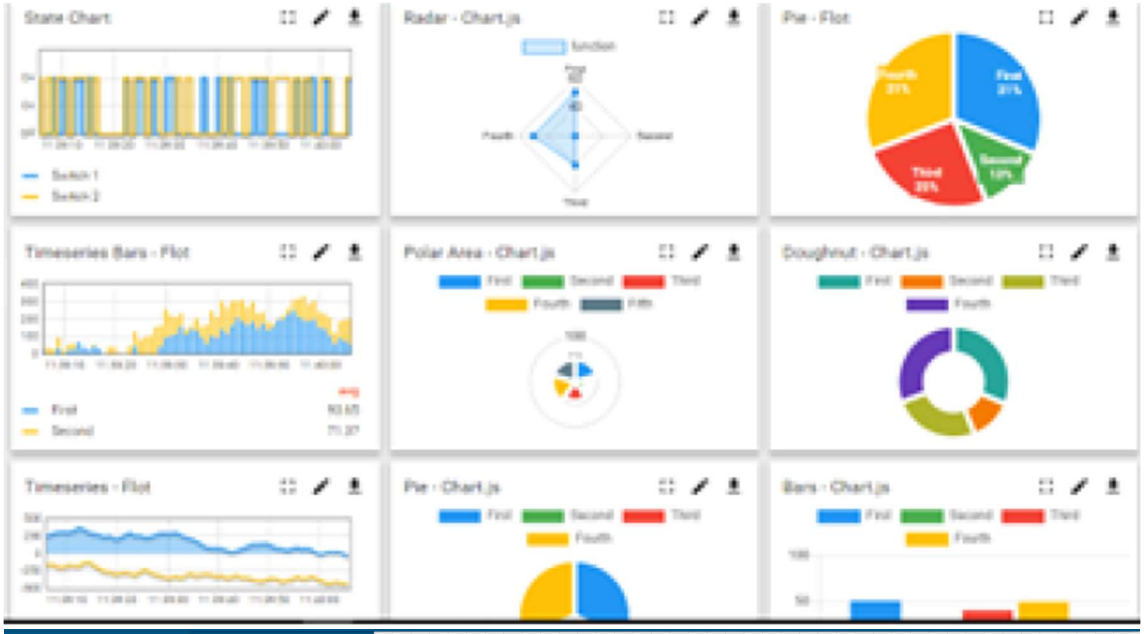
i. UCT IoT Platform (UCT Insight)

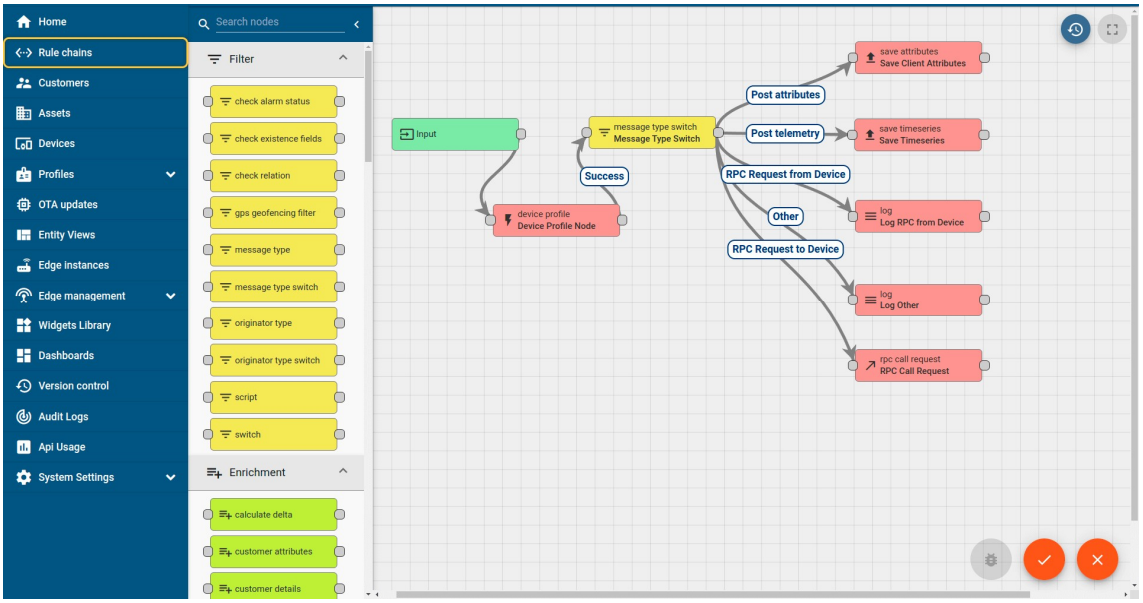
UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable "insight" for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine





ii. Smart Factory Platform (FACTORY WATCH)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

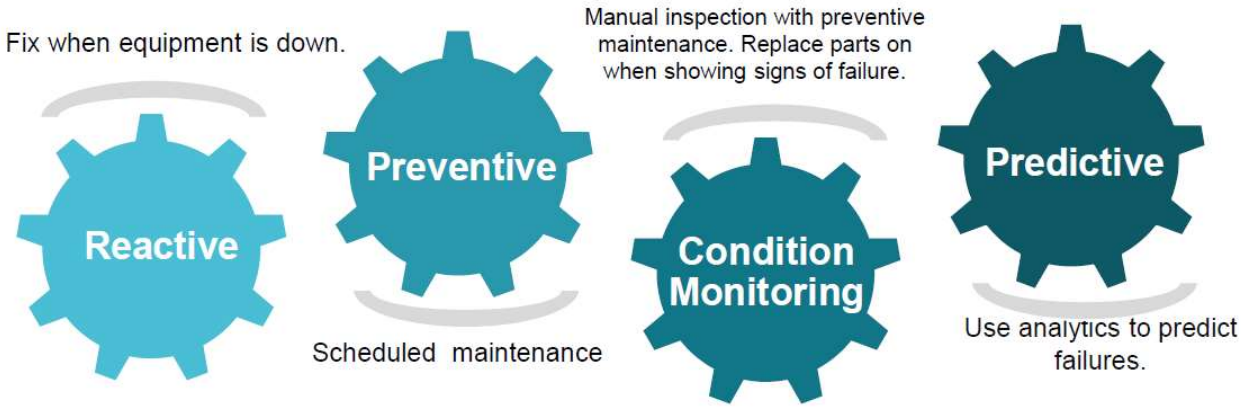


iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

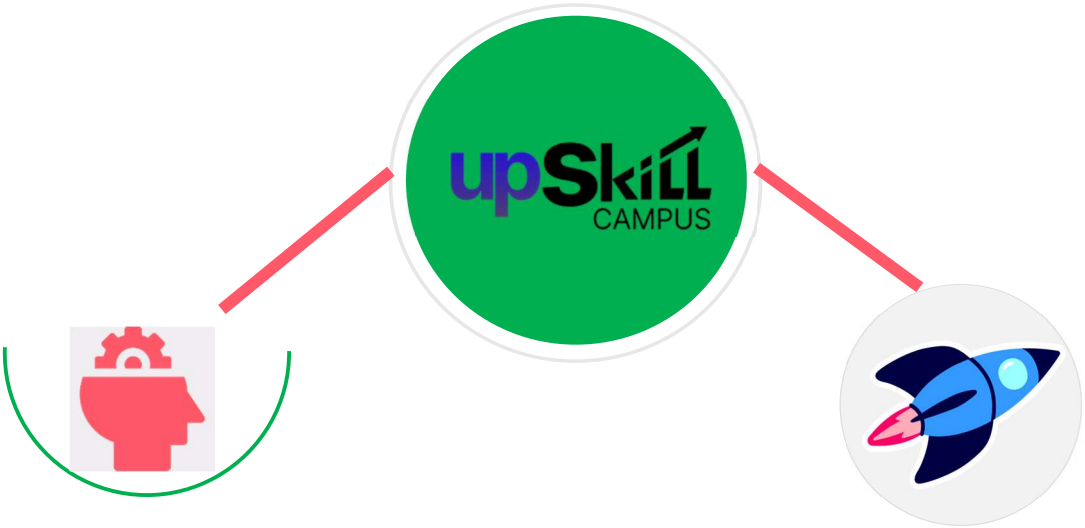
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

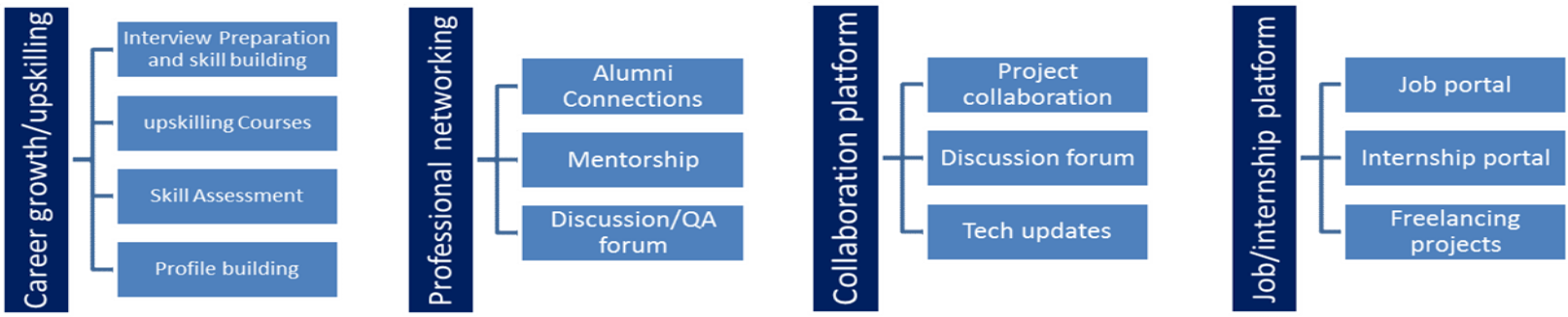
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

https://www.upskillcampus.com/



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- ☛ get practical experience of working in the industry.
- ☛ to solve real world problems.
- ☛ to have improved job prospects.
- ☛ to have Improved understanding of our field and its applications.
- ☛ to have Personal growth like better communication and problem solving.

2.5 Reference

Use the following references and replace/add URLs that you actually consulted:

1. Python Software Foundation, Python 3 documentation: https://docs.python.org/3/
2. Python Tkinter documentation: https://docs.python.org/3/library/tkinter.html
3. Python pathlib documentation: https://docs.python.org/3/library/pathlib.html
4. Python shutil documentation: https://docs.python.org/3/library/shutil.html
5. Project source code: SmartSort.py (the implementation supplied with this report).

2.6 Glossary

GUI	Graphical User Interface; visual controls through which the user interacts with the program.
Extension	The suffix of a filename, such as .pdf or .jpg, used here to determine a category.
Tkinter	Python's standard interface to the Tcl/Tk GUI toolkit.
Pathlib	Python module for representing and working with filesystem paths.

3 Problem Statement

- File Organizer:

Description: The file organizer is a Python project that helps users organize their files in a directory. It scans a specified directory, categorizes files based on their type (e.g., images, documents, videos), and moves them into respective folders.

Scope: The scope of this project involves designing a user interface to specify the directory to organize, implementing functions to identify file types and create folders, and developing a file-moving algorithm to organize files into the appropriate folders.

4 Existing and Proposed solution

4.1 4.1 Existing Approach and Limitations

A common approach is to manually create folders such as Documents, Images, Videos, and Code, then move files into the appropriate folder using the operating system's file manager. This approach is straightforward for a small number of files, but it becomes time-consuming for large mixed folders. The user must repeatedly inspect extensions, choose destinations, and check whether a file with the same name already exists.

Basic scripts can automate file movement, but a script that immediately moves files without showing a preview may cause confusion or unwanted changes. A safer utility should show the planned categories first, ask for confirmation, and keep a record of the movements.

4.2 4.2 Proposed Solution

The proposed solution is File Organizer, implemented as SmartSort. It is a Python desktop application with a Tkinter interface. The user chooses a folder and selects Preview files to see the filename, detected category, destination folder, and file size. When the user selects Organize files, the application presents a confirmation dialog summarizing the batch. After confirmation, each file is moved to a category folder.

The application chooses a new destination name if a duplicate filename exists, for example report (1).pdf, rather than overwriting the existing file. It records source and destination paths in a JSON history file, which is used by the Undo last batch feature. A result dialog reports how many files were moved and whether any errors occurred.

4.3 Code submission (Github link) : https://github.com/koyana-27/upskillcampus/blob/main/FileOrganizer.py

4.4 Report submission (Github link) : https://github.com/koyana-27/upskillcampus/blob/main/FileOrganizer.py

5 Proposed Design/ Model

File Organizer follows a simple event-driven design. The graphical interface receives user actions, helper functions identify categories and safe destination names, and filesystem operations perform the organization. A JSON file stores the movement history required by the undo feature.

5.1 High-Level Design

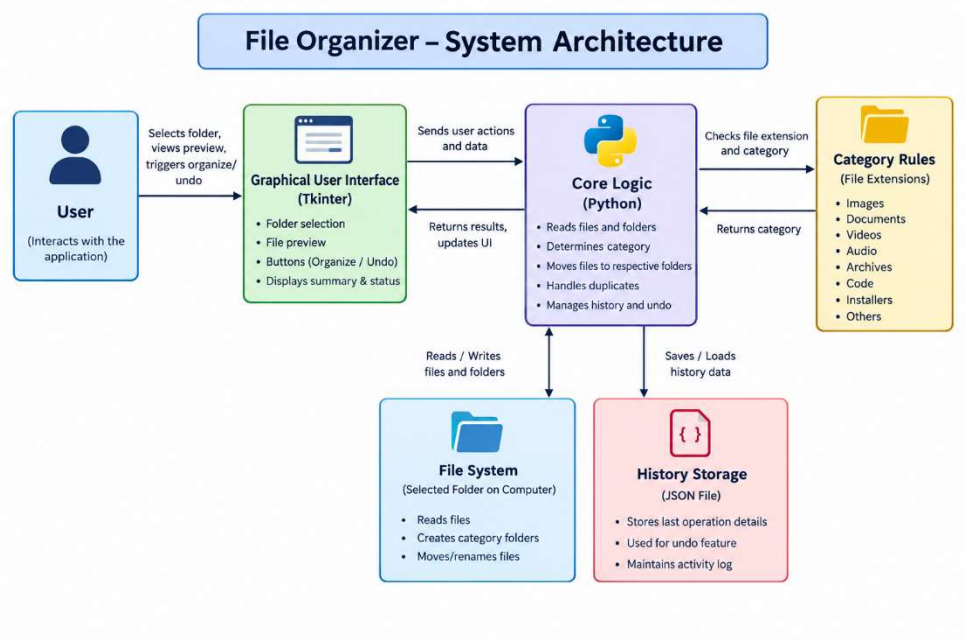


Figure 1: High-level architecture of SmartSort

Suggested diagram flow: User → Tkinter GUI → Folder validation and file preview → Extension-based classification → Confirmation → File movement and duplicate-name protection → JSON history and result summary.

5.2 Workflow

Start the application by running SmartSort.py.
Choose a folder using the Browse button or enter a valid folder path.
Preview the files. The program lists direct files and determines each category from its extension.
Review the proposed category and destination for each file.
Select Organize files and confirm the operation in the dialog.
The program creates category folders when required and moves files without overwriting existing files.
The application saves movement records to .smartsort_history.json and displays a result summary.
If required, select Undo last batch to attempt to return the most recent batch to its original locations.

5.3 User Interface and Main Modules

Component / Function	Purpose
Tkinter / ttk interface	Displays folder selection, preview table, status text, and action buttons.

category_for(path)	Maps a file extension to one of the supported categories; unknown extensions map to Others.
unique_destination(folder, filename)	Creates a non-conflicting destination filename when a duplicate exists.
_get_folder()	Validates that the selected path exists and is a directory.
_direct_files(folder)	Returns files directly inside the selected folder while excluding the history and temporary SmartSort files.
preview()	Lists files, calculates category counts and sizes, and displays the proposed organization.
organize()	Requests confirmation, creates category folders, moves files, and records history.
undo_last_batch()	Uses saved history to move the latest batch back to its original locations when possible.

Implementation note: The application is written for Python 3.9 or later and uses standard-library modules. Tkinter is normally included with Python on Windows.

6. Performance Test

Testing should confirm both the expected behavior and the safeguards of the application. The code includes checks for invalid folder paths, empty folders, duplicate destination names, user cancellation, file-operation errors, and missing or unreadable undo history. The test results below are a proposed test plan, not claims that these tests have already been executed. Fill in the Actual Result and Status columns after running the program.

6 6.1 Test Plan / Test Cases

Test ID	Test scenario	Expected result	Actual result / status
TC-01	Select a valid folder containing mixed file types.	Preview shows each direct file and its proposed category.	[FILL AFTER TEST]
TC-02	Choose an empty folder.	Application reports that there are no files to organize.	[FILL AFTER TEST]
TC-03	Enter an invalid folder path.	Application displays a validation error.	[FILL AFTER TEST]
TC-04	Use an unknown file extension.	File is classified under Others.	[FILL AFTER TEST]
TC-05	Destination already contains a same-named file.	A numbered alternative filename is selected; existing file is not overwritten.	[FILL AFTER TEST]
TC-06	Cancel the confirmation dialog.	No files are moved.	[FILL AFTER TEST]
TC-07	Organize a folder with mixed files.	Files are moved into the correct category folders and a summary is shown.	[FILL AFTER TEST]
TC-08	Undo the latest organization batch.	Recorded files are returned where possible; collisions are handled with a safe new name.	[FILL AFTER TEST]
TC-09	Open undo when no	Application reports that	[FILL AFTER TEST]

	history exists.	there is nothing to undo.	
TC-10	Include nested folders inside the selected folder.	Nested folders are not recursively processed.	[FILL AFTER TEST]

6.2 Test Procedure

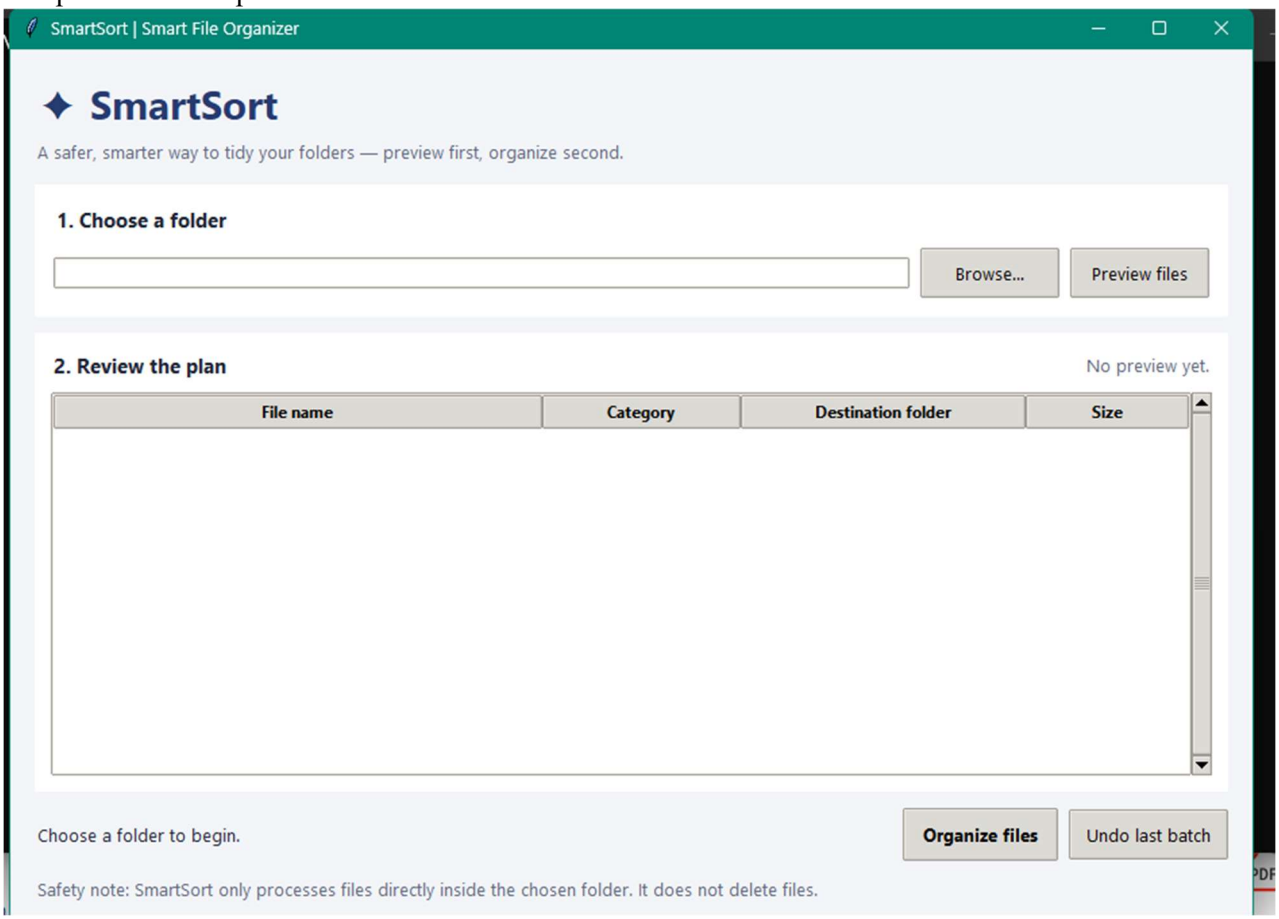
Create a temporary test folder and place sample files with different extensions inside it.
Run the application using the command: FileOrganizer.py.
Select the test folder and review the preview table before organizing.
Test cancellation first to confirm that the files remain in place.
Run the organization process and inspect the created category folders.
Repeat with a duplicate filename, an unknown extension, an empty folder, and an invalid path.
Use Undo last batch and compare the resulting files with the original test setup.
Record the actual outcome for each test case and attach screenshots if required by the internship coordinator.

6.3 Performance Outcome

The implementation is intended for lightweight, local folder organization. Its main operations are directory listing, extension lookup, folder creation, file movement, and writing a small JSON history file. Actual execution time depends on the number and size of files, storage speed, permissions, and the filesystem. No measured timing or benchmark results were supplied with the source code, so numerical performance claims should not be added unless they are measured. Enter the tested device details, number of files, elapsed time, and observed errors here.

Test environment: [RUN IN ALL PYTHON VERSION]

Sample size and elapsed time:



Overall test outcome: [PASS]

7 7. My Learnings

This project provided practical experience in translating a routine user problem into a small desktop application. I learned how to create a graphical interface using Tkinter and ttk, connect buttons to functions, and display information in a table. The project also demonstrated how a clear workflow—select, preview, confirm, execute, and summarize—can make a file operation easier for a user to understand.

Working with pathlib strengthened my understanding of filesystem paths, file extensions, and directory validation. The shutil module was used to move files, while json was used to store the source and destination paths needed for undo operations. The datetime module records when a batch was processed. I also gained experience with exception handling for common filesystem errors and with checking for destination-name conflicts before moving files.

An important lesson was that automation should include safeguards. Previewing the plan and asking for confirmation reduces the risk of unexpected changes. Avoiding overwrites protects existing files, and recording the operation makes it possible to reverse the latest batch in many cases. Testing also needs to cover invalid input, empty folders, duplicate filenames, cancellation, and failure cases rather than only the normal path.

These skills are useful for future work in Python automation, desktop utilities, and software development. In future projects, I would like to improve the test coverage, refine the interface, and add features based on user feedback. Personalize this section with the concepts you actually learned and any difficulties you faced during implementation.

8 8. Future Work Scope

The current implementation covers basic organization by file extension. Several enhancements could make the tool more flexible, robust, and suitable for larger collections of files:

Allow users to customize categories and file-extension mappings through settings.
Add an option to scan subfolders recursively, with safeguards against moving files into their own destinations.
Provide a dry-run export or a report file listing the proposed and completed movements.
Add progress indicators and cancellation support for large batches.
Improve undo reliability by validating the saved history and explaining when files have been changed or moved externally.
Add more detailed logs with timestamps, per-file outcomes, and error messages.
Package the application into a Windows executable so users can run it without manually launching Python.
Add automated unit tests for category mapping, duplicate-name handling, and history behavior.
Provide options to organize by file date, size, or user-defined rules in addition to extension.
Improve accessibility and interface responsiveness for different screen sizes.
These are proposed future improvements and are not implemented in the supplied SmartSort.py source.
Any feature completed later should be described separately from the current implementation.